

Hole in the Wall VR — Documentación del sistema

Guía para el equipo: qué hace cada parte, cómo se conecta todo y cómo levantarlo desde cero.

1. De qué se trata el proyecto

Réplica del juego de televisión *Hole in the Wall* (Human Tetris) aplicada a **rehabilitación motora**. Una pared con un agujero con forma de silueta avanza hacia el jugador, que tiene que adoptar esa postura con su cuerpo para pasar.

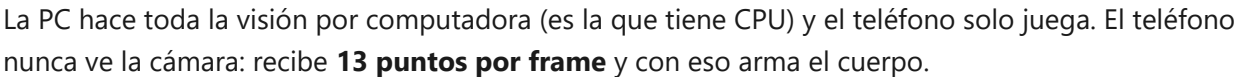
La diferencia con el juego original es el objetivo: acá las siluetas son **ejercicios de rehabilitación**. El paciente ve la pared en primera persona con un visor Cardboard, y una cámara común detecta su pose sin necesidad de sensores puestos encima. Eso trae dos consecuencias de diseño que conviene tener presentes al programar:

- **Las posturas tienen que ser alcanzables.** El rango de movimiento del paciente es limitado y variable. La tolerancia de acierto es configurable, y las medidas del cuerpo se toman del propio paciente antes de jugar.
- **Nada de sobresaltos ni movimientos bruscos de cámara.** Un jugador con dificultad motora no puede corregir rápido, y el mareo en VR empeora todo.

Estado actual: el juego funciona de punta a punta. Hay muros con agujeros con forma de pose, evaluación de la postura articulación por articulación, calibración de las medidas del paciente, modo sentado, un editor de poses y un espejo en la PC que replica la partida.

2. Arquitectura general

Tres piezas, que se hablan todas por el **mismo WebSocket**:



`cardboardgodot/ro1.gd` decide qué es cada instancia. **Es el mismo proyecto de Godot compilado de dos formas**, no dos proyectos:

Por qué un espejo y no mandar video. Se probaron las dos. Mandar imágenes cuesta 114 KB/s y, sobre todo, obliga a bajar la textura de la GPU a la CPU en cada cuadro, lo que frena el render del visor. El espejo manda **139 bytes a 30 Hz (~3 KB/s)** y no le cuesta nada al teléfono, porque la PC ya tiene la escena y los landmarks: solo le falta saber lo que no puede deducir.

Los mensajes

Todos por el mismo socket. El servidor **retransmite** entre clientes lo que no es suyo, y guarda `cfg` y `poses` para repetírselas a quien se conecte después.

Mensaje	Va	Para qué
landmarks	servidor → todos	Los 13 puntos del cuerpo, 30 fps
est	visor → espejo	Estado de la partida (ver abajo)
cmd	espejo/servidor → visor	Un botón apretado desde la PC
cfg	cualquiera → todos	Modo sentado, alza de cadera, apertura
poses	espejo → todos	Lista de poses editada
binario	visor → servidor	Cuadro JPEG (respaldo, apagado)

Landmarks, ~30 veces por segundo:

```
{
  "landmarks": [
    { "id": 0, "name": "nose", "x": 0.505, "y": 0.727, "z": -0.952 },
    { "id": 11, "name": "left_shoulder", "x": 0.695, "y": 0.963, "z": -0.327 }
  ],
  "timestamp": 1785786061.45
}
```

- **x**, **y** → normalizados respecto del tamaño de la imagen.
- **y = 0** es **arriba** de la imagen (convención de imagen, no de 3D).
- **z** → profundidad relativa que estima MediaPipe. **No se usa** (`LandmarkDepth = 0`): es ruidosa y en metros de mundo empujaba al jugador fuera del mapa.

⚠ **x** e **y** **PUEDEN SALIRSE del rango [0,1]**. Cuando el cuerpo se sale del encuadre, MediaPipe extrapola. Sentado y cerca de la cámara, los tobillos llegan a **y ≈ 2.4**. Es el error más fácil de cometer con estos datos, y la razón de que todo el sistema trabaje con **proporciones** y no con posiciones crudas (ver §5).

⚠ **x** se normaliza por el **ANCHO** de la imagen e **y** por el **ALTO**. O sea que las medidas horizontales vienen deformadas por el *aspect ratio* de la cámara. Para el juego no importa mientras la calibración se tome en el mismo espacio en que después se compara al jugador, porque la deformación se cancela; sí importa si alguna vez se quiere que el agujero tenga proporciones físicas correctas.

Estado de la partida, del visor al espejo:

```
{
  "est": 2,
  "cab": [0.75, 1.75, 2.33, 0, 0, 0, 1],
  "cal": [0.253, 0.142, 0.364, 0.38, 0.752, 0.72, 0.405],
  "muros": [[7, 3, -4.5]],
  "ok": 5, "fail": 2
}
```

`cab` es posición + cuaternión de la cabeza; `ca1` las siete proporciones calibradas; `muros` es `[id, índice de pose, z]` por cada muro vivo. Los contadores van como números y no como eventos sueltos: si se pierde un mensaje, el siguiente igual trae el salto y el flash no se pierde.

3. Los archivos principales

3.1 Servidor — `servidor/servidor.py`

Único archivo del lado PC. Corre dos cosas en paralelo:

- **Hilo de MediaPipe** → abre la cámara, detecta la pose, guarda las posiciones en `pos_nodos`, y muestra la ventana de previsualización con los puntos dibujados encima. Desde esa ventana también se maneja el juego (§6.4).
- **Loop asincio** → cada 1/30 s manda los landmarks a todos los clientes, y retransmite entre ellos lo que se mandan.

Usa la **API Tasks** de MediaPipe, no la vieja `mp.solutions`, que fue **eliminada en MediaPipe 1.0**. Por eso necesita `servidor/pose_landmarker_lite.task` (ya está en el repo). Si ves ejemplos en internet con `mp.solutions.pose`, son de una versión anterior y no funcionan.

El frame se invierte con `cv2.flip` para que la vista previa sea tipo espejo. Eso tiene una consecuencia: MediaPipe etiqueta izquierda/derecha según el lado de la **imagen**, así que los ids quedan cruzados respecto del jugador. Lo compensa `Figuras.IDS_ESPEJADOS`.

3.2 El juego — `cardboardgodot/muro/`

Archivo	Qué hace
<code>figuras.gd</code>	Las poses y las proporciones del cuerpo. Reconstruye cada esqueleto, define el modo sentado y serializa las poses para el editor.
<code>muro.gd</code>	Un muro: talla el agujero por CSG, dibuja los dos esqueletos 2D y evalúa la postura.
<code>gestor_juego.gd</code>	El ciclo del juego, la calibración en T, el flash de resultado y todo lo del espejo.
<code>demo_muros.gd</code>	Demo ejecutable de los muros sin servidor ni VR (F6).

3.3 Cliente y escena — `cardboardgodot/example/`

Archivo	Qué hace
<code>example_scene.gd</code>	El cliente WebSocket. Se conecta, recibe landmarks, arma el esqueleto, lo suaviza y lo normaliza. <code>gestor_juego.gd</code> lo extiende.
<code>Player.tscn</code>	El jugador: un <code>CharacterBody3D</code> con la cámara Cardboard adentro.
<code>player.gd</code>	Movimiento con teclado. (<i>Heredado del ejemplo del addon; hoy no se usa.</i>)

La escena principal es `escenario/Escenario.tscn`: la piletta, el agua, el piso invisible y el Player, girado 180° y parado al borde de la piletta de espaldas al agua.

`example_scene.gd` hace cinco cosas:

1. **Descubrimiento del servidor.** Escanea la subred en la que está el propio teléfono probando `.1 ... .20` en el puerto 8765. Saltea las subredes link-local `169.254.x`, que Windows expone de a varias y son inalcanzables: cada sondeo agotaba el timeout entero y encontrar el servidor tardaba más de un minuto.
2. **Armado del esqueleto** por código (`_armar_esqueleto`).
3. **Suavizado (filtro One Euro).** Los datos de MediaPipe tiemblan. Un promedio simple obliga a elegir entre tembleque (poco filtro) o retardo (mucho), y en VR el retardo mareaba. El One Euro **varía el filtrado según la velocidad**: filtra fuerte cuando el punto está casi quieto —que es cuando el ruido se nota— y afloja al moverse rápido.
4. **Normalización de la nube** (`NormalizarNube`). Escala el cuerpo para que el torso mida lo que el del personaje y lo apoya en el piso por los tobillos. Sin esto los puntos quedan en coordenadas del **encuadre**: el esqueleto se dibuja del tamaño que ocupa la persona en la imagen, y acercarse a la cámara le mete medio cuerpo bajo el piso. Se hace **una sola vez**, así el cuerpo que se VE es exactamente el que se COMPARA.
5. **FollowHead**. El landmark 0 (la nariz) controla el punto de vista. De él se toma el corrimiento lateral y **cuánto se agachó** —medido como qué tanto sobresale la cabeza sobre la cadera, en torsos—, nunca la `y` cruda, que depende del encuadre y dejaba la cámara a la altura del pecho.

3.4 El espejo — `cardboardgodot/espejo/panel.gd`

El panel de control de la PC: modo sentado/parado, alza de cadera, apertura de piernas y el editor de poses. Se abre con **TAB**.

3.5 El addon de Cardboard — `cardboardgodot/addons/cardboard_vr/`

Archivo	Qué hace
<code>scripts/cardboard_vr_camera.gd</code>	El corazón del VR. Crea dos <code>SubViewport</code> (uno por ojo), aplica separación y convergencia, y lee el giroscopio.
<code>shaders/LensDistortion.gdshader</code>	Corrección de distorsión de lente (Brown-Conrady, $k1/k2$).
<code>scripts/calibration_menu.gd</code>	Menú libre de calibración.
<code>scripts/calibration_wizard.gd</code>	Asistente guiado en dos fases.

⚠ **Los pivotes de ojo cuelgan de los `SubViewport`, no del jugador**, así que su rotación es **absoluta del mundo**. Por eso `recenter()` no los pone en cero sino en el `yaw` con el que quedó puesto el Player en la escena: girar el nodo Player no alcanzaba, y la vista arrancaba mirando al `-z` del mundo sin importar cómo estuviera orientado el cuerpo.

4. Cómo funciona el juego

4.1 Las poses

Las 10 poses del storyboard viven en `Figuras.storyboard()`. De cada una se usa solo la **dirección** de cada segmento: los **largos** salen de las proporciones del cuerpo. Así la pose sigue siendo la que se dibujó y además le entra un cuerpo real.

Hizo falta porque las figuras del storyboard venían con brazos y piernas cortos: los tobillos caían a 1.11 torsos de la cadera cuando en un cuerpo real caen a 1.47. Para llegar al objetivo había que levantar los pies del piso.

Toda figura se baja hasta **apoyar los tobillos**, así agacharse baja la cadera en vez de levantar los pies.

4.2 El muro

El agujero se talla por resta booleana (CSG): a la caja se le restan los huesos (cajas orientadas), **un cilindro en cada articulación** para redondear las esquinas internas, uno más grande para la cabeza, y **dos polígonos rellenos** —el torso y el triángulo del cuello—. Sin esos polígonos quedaban islas de pared flotando en el medio del agujero, porque los huesos son tiras y no rellenan las zonas que encierran.

Sobre la cara que mira al jugador se dibujan **dos esqueletos 2D superpuestos**: el objetivo (celeste, translúcido) y el del jugador (amarillo, opaco), con las articulaciones en verde o rojo. Los dos se desvanecen a partir de 3.5 m para que la imagen de atravesar el agujero quede limpia.

El muro **no frena**: avisa el resultado al cruzar el plano del jugador, sigue de largo y se borra 12 m después. El resultado lo da un **flash de cámara** verde o rojo sobre la imagen estéreo ya compuesta.

4.3 La evaluación

Cada articulación tiene que caer a menos de `tolerancia` (0.20 m) de **su** nodo objetivo. No alcanza con entrar por el agujero: con el criterio viejo, de contención, quedarse hecho un palito pasaba casi cualquier pose.

Medido sobre las 10 poses:

tolerancia	palitos que pasan	poses que aprueban la suya	falsos positivos cruzados
0.30	1 de 10	10	8
0.25	0	10	8
0.20	0	10	2
0.16	0	10	0

Se eligió 0.20: cruzarse de pose exige adoptar **otra pose válida entera**, cosa que nadie hace por accidente, mientras que el palito era quedarse quieto. Con 0.16 no hay cruces pero empieza a pelearse con el ruido de MediaPipe.

4.4 Calibración del paciente (botón 3)

Con el jugador **parado en T**, toma sus medidas y reescribe las proporciones de las figuras, así los muros quedan a su medida.

- Promedia por **mediana sobre 2 segundos**, no un solo frame: un brazo mal detectado se comía la calibración entera.
- Mide **relaciones contra el torso**, nunca tamaños absolutos, así deja de importar a qué distancia de la cámara esté parado.
- **Valida antes de aceptar**. Cada medida tiene que caer dentro de $\pm 35\%$ de la de referencia, y además se chequean dos invariantes anatómicos: los dos segmentos de un miembro tienen que ser parecidos entre sí. Hace falta porque una captura mala puede tener todas las medidas dentro del margen y aun así describir un cuerpo imposible — una toma fallida real dio antebrazo 0.21 contra brazo 0.50, y pantorrilla 0.78 contra muslo 0.42. Si falla, avisa en pantalla y no toca nada: **quien calibra tiene el visor puesto y no puede ver lo que capturó**.

Proporciones de referencia (en múltiplos del torso, medidas sobre una persona real, 120 frames, dispersión p10–p90 bajo el 2%):

Medida	Valor
Medio ancho de hombros	0.253
Medio ancho de cadera	0.142
Hombro → codo	0.364
Codo → muñeca	0.380
Cadera → rodilla	0.752
Rodilla → tobillo	0.720
Cuello → nariz	0.405

4.5 Modo sentado

Para pacientes que no pueden jugar de pie. **No hay poses aparte**: cambian tres cosas en la reconstrucción.

1. Las piernas van **juntas y rectas** (apertura 0).
2. La vertical se ancla en la **cadera** en lugar de los tobillos, que estando sentado quedan bajo el escritorio y MediaPipe los extrapola sin ninguna base. `a1za_cadera` sube la figura sin tocar las poses.
3. **Rodillas y tobillos dejan de contar** para aprobar (quedan 9 de 13 articulaciones). De frente, un muslo apuntando a la cámara se ve escorzado y no hay forma de que coincida con un objetivo vertical; exigirlo volvía el modo injugable.

	separación de tobillos	articulaciones que cuentan
Parado	0.898	13/13
Sentado	0.155	9/13

4.6 Editor de poses

En el espejo, con **TAB**. Permite agregar, duplicar, borrar y renombrar poses, y arrastrar las articulaciones con el mouse.

Dibuja el esqueleto **reconstruido** —con los largos anatómicos y, si está activo, las piernas juntas del modo sentado— y no los puntos crudos del storyboard. Es importante entenderlo para usarlo: arrastrar un codo cambia el **ángulo** del brazo, no su largo.

Los cambios se aplican al instante en la PC y se publican al visor con "Aplicar y enviar". Las 10 poses ocupan 1286 bytes.

5. Los tres sistemas de coordenadas

Es lo que más confunde al leer el código, así que conviene tenerlo claro:

Espacio	Unidad	Origen	Dónde
Imagen	normalizada 0–1	esquina sup. izq.	lo que manda MediaPipe
Figura	arbitraria (torso = 1.40)	cadera en <code>y = -0.35</code>	cómo se escriben las poses
Muro	metros	piso en <code>y = 0</code>	dónde se compara todo

`Figuras.a_metros()` pasa de figura a muro. La escala sale de que la pose de pie mida `ALTURA` (1.8 m): da **0.39 m por unidad**, con lo que el torso mide 0.546 m y el tobillo apoya a 0.071 m del piso.

6. Cómo ejecutar todo

6.1 Requisitos

PC: Python 3.13, Godot **4.4.1**, Android SDK con `adb`, cámara web.

Teléfono: Android con giroscopio, depuración USB activada, visor Cardboard y control bluetooth.

6.2 Instalar dependencias de Python

```
python -m pip install mediapipe websockets opencv-contrib-python numpy
```

El modelo `pose_landmarker_lite.task` ya viene en el repo. Si faltara:

```
curl -L -o servidor/pose_landmarker_lite.task "https://storage.googleapis.com/mediapipe-models/pose_landmarker_lite.task"
```


6.3 Crear la red WiFi en Windows 11

Este es el paso que más problemas da. La PC y el teléfono tienen que estar en la misma red; lo más simple es que la PC cree la suya.

1. **Configuración** → **Red e Internet** → **Zona con cobertura inalámbrica móvil** (*Mobile hotspot*).
2. Elegí **Wi-Fi** en "Compartir mi conexión a Internet desde".
3. **Editar** → poné nombre y contraseña.
4. Activá el interruptor.
5. En el teléfono, conectate a esa red.

Windows le asigna a la PC siempre la IP `192.168.137.1`.

⚠ **Android se desconecta solo del hotspot.** Como esa red muchas veces no da internet, el teléfono se cambia solo a otra WiFi y deja de ver a la PC. Es la causa número uno de "dejó de andar".

`bash adb shell ip addr show wlan0` Tiene que decir `192.168.137.x`.

⚠ **El Firewall de Windows puede bloquear el puerto 8765.** La primera vez, Windows pregunta si permitís el acceso: **aceptá para redes privadas**.

6.4 Arrancar (la forma corta)

Doble clic en `EspejoPC.exe`. Hace todo solo:

1. Mata lo que esté ocupando el puerto 8765.
2. Levanta `servidor/servidor.py` con consola visible — si no puede abrir la cámara o le falta el modelo, lo dice ahí.
3. Abre su ventana de 1280×720.
4. Escanea la red y se conecta.

Después se abre la app en el teléfono y listo.

Por qué mata el puerto primero. Un servidor a medio morir sigue escuchando aunque no haya podido abrir la cámara —por ejemplo si al arrancar otro se la había quedado— y desde afuera es indistinguible de uno sano: el puerto contesta, pero no hay ventana de cámara ni landmarks. Reusarlo dejaba el juego sin cuerpo. Ojo que mata **cualquier** proceso en 8765, no solo el servidor.

Levantarlo a mano sigue andando:

```
cd servidor && python servidor.py
```

6.5 Compilar

```
rm -rf cardboardgodot/android/build/build/
```

```
"C:/Program Files (x86)/Godot/Godot_v4.4.1-stable_win64.exe" --headless --path cardboardgodot --export-c
```

```
adb install -r cardboardgodot/cardboardtest.apk
```

```
adb shell am start -n "com.example.cardboardgodot/com.godot.game.GodotApp"
```

El espejo:

```
"C:/Program Files (x86)/Godot/Godot_v4.4.1-stable_win64.exe" --headless --path cardboardgodot --export-c
```

Borrá siempre `android/build/build/` **antes de exportar**. El estado incremental viejo hace fallar el build con `processStandardDebugMainManifest`.

Si agregás una clase nueva con `class_name` sin usar el editor, hay que reescanear o Godot no la encuentra: `godot --headless --path cardboardgodot --import`

6.6 Controles

	Visor (control bluetooth)	PC (espejo o ventana de cámara)
Empezar / parar	Botón 2	<code>Enter</code> o <code>Espacio</code>
Calibrar en T	Botón 3	<code>C</code>
Recentrar la vista	Botón 9	<code>R</code>
Menú de calibración	Botón 1	—
Asistente guiado	Botón 10	—
Cambiar punto de vista	—	<code>V</code>
Panel y editor de poses	—	<code>TAB</code>

Parar con el botón 2 vuelve a la bienvenida sin cerrar la app, para poder recalibrar.

6.7 Verificar que conectó

```
adb logcat -s godot | findstr landmarks
```

```
[rol] VISOR (telefono)
[landmarks] Buscando servidor en 192.168.137.X:8765 ...
[landmarks] Conectado a ws://192.168.137.1:8765
[landmarks] Recibiendo (30 frames)
```

7. Calibración del visor

Cada visor Cardboard y cada persona son distintos. Se maneja con el control.

- **Asistente guiado (botón 10), fase 1 — Distancia interpupilar.** Ajustar hasta ver una sola figura nítida, sin doble.
- **Fase 2 — Distorsión de lente.** Aparece una cuadrícula; ajustar k1 y k2 hasta que las líneas se vean rectas, sobre todo en los bordes.

Se guarda solo en `user://cardboard_calibration.cfg`.

Valores actualmente guardados

Parámetro	Valor
Separación de ojos	0.005
Altura de ojos	1.75 m
Ángulo de convergencia	8.9°
Sensibilidad giroscopio	0.022

 **La separación de ojos está prácticamente en cero.** 0.005 son ± 5 mm (10 mm en total) cuando la distancia interpupilar real ronda los 63 mm. Con eso los dos ojos ven casi la misma imagen: **no hay paralaje estereoscópica**, y sin paralaje la profundidad no se percibe — el muro que se acerca, la piletta y el esqueleto se ven planos. También hace que alejar un cartel no se note: lo único que registra el ojo es cuánto del campo visual ocupa. Si no fue a propósito, conviene subirla hacia ~ 0.031 .

8. Diagnóstico de problemas

Síntoma	Causa probable	Qué hacer
Sin servidor en ...	El teléfono se cambió de red	<code>adb shell ip addr show wlan0</code>
El espejo dice que hay servidor pero no hay cámara	Servidor zombi en el puerto	Lo mata solo al arrancar; a mano, <code>taskkill /PID <pid> /F</code>
<code>adb devices</code> dice <code>unauthorized</code>	Falta autorizar la clave RSA	Desbloqueá la pantalla: el diálogo no aparece con el teléfono bloqueado
<code>adb devices</code> vacío	Modo USB en "sin transferencia"	Notificación de USB → Transferencia de archivos
No se ve el esqueleto	La cámara no te detecta	Mirar la ventana de MediaPipe
El esqueleto crece al acercarse	<code>NormalizarNube</code> apagado	Prenderlo
La cámara no enciende (sin LED)	<code>VideoCapture(0)</code> tomó una cámara virtual	Cerrar OBS o DroidCam
Todo tiembla	Filtro flojo	Bajar <code>MinCutoff</code> a 0.8
Se siente pastoso	Filtro muy fuerte	Subir <code>MinCutoff</code> a 2.0 y <code>SmoothBeta</code> a 0.1
Se ve doble / no fusiona	Falta calibrar	Botón 10
La vista quedó torcida	Deriva del giroscopio	Botón 9
Las poses de un brazo piden el brazo contrario	Lateralidad invertida	<code>Figuras.IDS_ESPEJADOS</code>

9. Lo que falta hacer

1. **Registro de sesión**, para poder seguir el progreso del paciente.
2. **Hornear los muros a malla estática**. El CSG recalcula en CPU; para el build final conviene precalcularlos.
3. **Persistir las poses editadas** en disco: hoy viven en memoria y se pierden al cerrar.
4. **Persistir la calibración del paciente**, hoy también en memoria.
5. **Definir las siluetas de ejercicio** junto a quien sepa de rehabilitación.

Detalles conocidos, sin resolver

- `player.gd` referencia acciones de input que no existen (`jump` , `up` , `down` , `left` , `right`) y llena el log de errores cada frame. No rompe nada.
- Las escenas de `cuerpo/` apuntan a rutas viejas. No se usan.
- La separación de ojos casi en cero (\$7).
- El muro frena su evaluación en el plano del jugador, así que al llegar la cara queda a unos 20 cm de los ojos. Se corrige moviendo `Z_PLANO` .

10. Referencia rápida de parámetros

Cliente de landmarks (`example_scene.gd`)

Parámetro	Default	Para qué
<code>ServerPort</code>	8765	Puerto del servidor
<code>SubnetBase</code>	<code>192.168.137.</code>	Subred del hotspot
<code>CloudDistance</code>	0.0	La nube va centrada EN el jugador
<code>CloudScale</code>	2.0	Metros por unidad normalizada
<code>NormalizarNube</code>	true	Escala al torso y apoya en el piso
<code>LandmarkDepth</code>	0.0	Cuánta <code>z</code> de MediaPipe se aplica
<code>FollowHead</code>	true	La nariz controla el punto de vista
<code>MinCutoff</code>	1.2	Suavizado: más bajo = más suave
<code>SmoothBeta</code>	0.04	Suavizado: más alto = más reactivo
<code>ShowDebugSkeleton</code>	false	Segundo esqueleto de debug
<code>TransmitirVista</code>	false	Respaldo por streaming de imágenes

Juego (`gestor_juego.gd`)

Parámetro	Default	Para qué
<code>velocidad_muro</code>	1.5 m/s	Velocidad de los muros
<code>spawn_cada</code>	8.0 s	Igual al tiempo de viaje: van de a uno
<code>escala_cuerpo</code>	1.0	Ajuste fino del tamaño del cuerpo
<code>segundos_calibracion</code>	2.0	Ventana de la calibración en T
<code>duracion_flash</code>	0.9 s	Flash de resultado
<code>EstadoFps</code>	30	Frecuencia del estado al espejo

Muro (`muro.gd`)

Parámetro	Default	Para qué
<code>ancho / alto</code>	8.0 / 3.0 m	Cubre el ancho de la piletta
<code>grosor</code>	0.20 m	Ancho del hueco por hueso
<code>holgura_cabeza</code>	1.6	Cuánto más grande que el cráneo
<code>tolerancia</code>	0.20 m	Radio de acierto por articulación
<code>distancia_desvanecer</code>	3.5 m	Dónde empiezan a desaparecer los esqueletos
<code>distancia_salida</code>	12 m	Cuánto sigue de largo antes de borrarse

Figuras (figuras.gd)

Parámetro	Default	Para qué
ALTURA	1.8 m	Altura del personaje
apertura_piernas	0.55	Cuánto se conserva de la apertura del dibujo
sentado	false	Modo sentado
alza_cadera	0.15 torsos	Cuánto se levanta la figura sentada
IDS_ESPEJADOS	true	Compensa el cv2.flip del servidor